

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI and application Development and jira

Practical –6

Roll No.:T050	Name: Aashu Yadav
Class:TYIT	Batch:1
Date of Assignment:19-8-2026	Date/Time of Submission:19-8-2026

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.99], [0.01]],
    state_names={"Burglary": ["False", "True"]}
)
# P(Earthquake)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05],
        [0.001, 0.29, 0.94, 0.95]
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10],
        [0.05, 0.90]
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30],
        [0.01, 0.70]
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
```

```

    cpd_alarm,
    cpd_john,
    cpd_mary
)
print("Is Bayesian Network valid?", model.check_model())
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={

```

```

        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and
MaryCalls=True:")
print(result)
print("Aashu T050")

```

Output:-

```

Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----+
| Burglary | phi(Burglary) |
+-----+
| Burglary(False) | 0.1998 |
+-----+
| Burglary(True) | 0.8002 |
+-----+
Aashu T050

```

-----different Example-----

Code:-

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Create Bayesian Network
model = DiscreteBayesianNetwork([
    ('Rain', 'WetGrass')
])
# CPD for Rain
cpd_rain = TabularCPD(
    variable='Rain',
    variable_card=2,
    values=[[0.7],
            [0.3]],
    state_names={
        'Rain': ['No', 'Yes']
    }
)
# CPD for WetGrass given Rain
cpd_wetgrass = TabularCPD(
    variable='WetGrass',
    variable_card=2,
    values=[
        [0.9, 0.2], # WetGrass = No
        [0.1, 0.8] # WetGrass = Yes
    ],
    evidence=['Rain'],
    evidence_card=[2],
    state_names={

```

```

        'WetGrass': ['No', 'Yes'],
        'Rain': ['No', 'Yes']
    }
)
# Add CPDs to the network
model.add_cpds(cpd_rain, cpd_wetgrass)
# Check the model
print("Model is valid:", model.check_model())
# Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=['Rain'],
    evidence={'WetGrass': 'Yes'}
)
# Display posterior probability
print("\nPosterior Probability:")
print(result)
print("Aashu T050")

```

Output:-

```

Model is valid: True

Posterior Probability:
+-----+
| Rain | phi(Rain) |
+-----+
| Rain(No) | 0.2258 |
+-----+
| Rain(Yes) | 0.7742 |
+-----+
Aashu T050

```

2. Write a program to compute Posterior Probability using BayesianNetworks (Burglar|Alarm = True) .

Code:-

```
# Posterior Probability using Bayesian Network
# Query: P(Burgary | Alarm = True)
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burgary", "Alarm"),
    ("Earthquake", "Alarm")
])
# Define probability of Burgary
cpd_burgary = TabularCPD(
    variable="Burgary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burgary": ["False", "True"]}
)
# Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# Define probability of Alarm
# P(Alarm | Burgary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
```

```
values=[
    [0.999, 0.71, 0.06, 0.05], # Alarm = False
    [0.001, 0.29, 0.94, 0.95] # Alarm = True
],
evidence=["Burgary", "Earthquake"],
evidence_card=[2, 2],
state_names={
    "Alarm": ["False", "True"],
    "Burgary": ["False", "True"],
    "Earthquake": ["False", "True"]
}
)
# Add CPDs to model
model.add_cpds(
    cpd_burgary,
    cpd_earthquake,
    cpd_alarm
)
# Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burgary"],
    evidence={"Alarm": "True"}
)
print("Posterior Probability:")
print("P(Burgary | Alarm = True)")
print(result)
print("Aashu T050")
```

Output:-

```
Is Bayesian Network valid? True
Posterior Probability:
P(Burgary | Alarm = True)
+-----+-----+
| Burgary | phi(Burgary) |
+-----+-----+
| Burgary(False) | 0.6264 |
+-----+-----+
| Burgary(True) | 0.3736 |
+-----+-----+
Aashu T050
```

-----different Example -----

Code:-

```
# Posterior Probability using Bayesian Network
# Query: P(Rain | WetGrass = True)

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Create Bayesian Network structure
```

```
model = DiscreteBayesianNetwork([
    ("Rain", "WetGrass")
])
# Define probability of Rain
cpd_rain = TabularCPD(
    variable="Rain",
    variable_card=2,
    values=[[0.8], [0.2]],
    state_names={"Rain": ["False", "True"]}
```

```

)
# Define probability of WetGrass
# P(WetGrass | Rain)
cpd_wetgrass = TabularCPD(
    variable="WetGrass",
    variable_card=2,
    values=[
        [0.9, 0.1], # WetGrass = False
        [0.1, 0.9] # WetGrass = True
    ],
    evidence=["Rain"],
    evidence_card=[2],
    state_names={
        "WetGrass": ["False", "True"],
        "Rain": ["False", "True"]
    }
)
# Add CPDs to model
model.add_cpds(
    cpd_rain,
    cpd_wetgrass
)
# Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())

# Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Rain"],
    evidence={"WetGrass": "True"}
)
# Display posterior probability
print("Posterior Probability:")
print("P(Rain | WetGrass = True)")
print(result)
print("Aashu T050")

```

Output:-

```

Is Bayesian Network valid? True
Posterior Probability:
P(Rain | WetGrass = True)
+-----+-----+
| Rain      | phi(Rain) |
+-----+-----+
| Rain(False) | 0.3077 |
+-----+-----+
| Rain(True)  | 0.6923 |
+-----+-----+
Aashu T050

```